

png"

```
ffmpeg -i "${sInputFileName}" \
-lavfi "[0:a:0]
showwavespic=s=600x120:
    split_channels=1:
    colors=yellow|red:
    scale=sqrt" -update 1 \
"${sOutputFileName}"
```

Audio normalisation and compression

Some multimedia files suffer from low audio volume. Rather than blindly increasing the volume, you should first use the 'volumedetect' filter to detect the highest decibel level and then use the 'volume' filter to expand or maximise the waveform accordingly.

```
ffmpeg -i low.mp3 -af "volumedetect" \
-f null -
```

```
ffmpeg -i low.mp3 \
-af 'volume=17dB' -f ogg \
normalized.ogg
```

In this example, the 'volumedetect' filter found that the maximum volume was at -17dB. To bring it to 0dB, you can set the 'volume' filter to 17dB. This expands the waveform to the maximum safe level. If we expand the waveform beyond the maximum value (17dB) identified by the 'volumedetect' filter, some of the peaks of the waveform will start to get chopped off. Chopped-off waveform means a permanent loss of some audio content.

Normalisation may not always raise volume levels. Imagine a conversation recorded on a microphone placed on a desk. If someone jerks the desk, the microphone will wobble and introduce high-volume spikes into the recording. Since these spikes set the bar, the recording software will encode the conversation with low volume. Imagine another case where there is a speaker who is close to the microphone and his voice is recorded normally while that of someone in the audience is barely

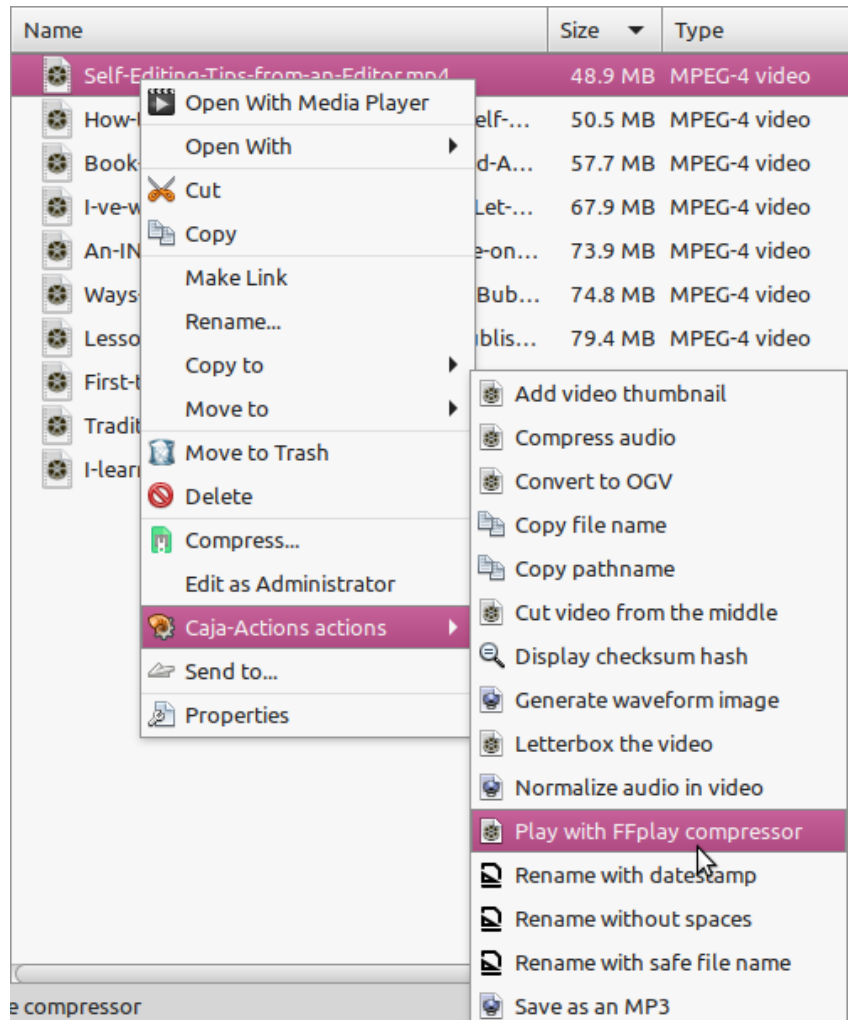


Figure 1: Caja context menus help automate FFmpeg tasks

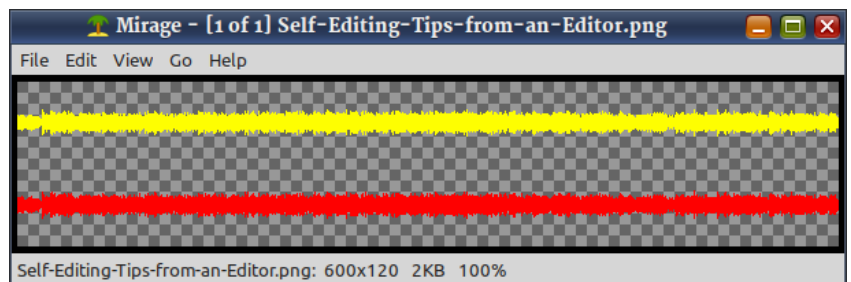


Figure 2: The output of the `showwavespic` filter can be saved as an image

discernible. The low-volume sections caused by the audience member cannot be expanded unless the audio of the first speaker is compressed.

Rather than considering the entire audio and not being able to normalise it, the smarter approach would be to

take smaller chunks of the audio and dynamically expand or compress their waveforms. This is known as *dynamic audio normalisation* and *dynamic audio compression*, respectively.

The 'dynaudnorm' filter can perform both functions but the default